AI Tool(s) Used Model: Gemini (Gemini 3.1 Pro)

Scope of Usage 1: Bucket Tool (Scanline Flood Fill)

Location: app.js, lines 275-383

Prompt: "Bucket Tool: Filling a closed area with a color"

Response:

```javascript
// ========================================
// 油漆桶演算法 (Scanline Flood Fill)
// ========================================
function floodFill(startX, startY) {
    const canvas = canvases[activeLayerIndex];
    const ctx = contexts[activeLayerIndex];
    const w = canvas.width;
    const h = canvas.height;

    // 取得畫布全部的像素資料 (一維陣列：[R, G, B, A, R, G, B, A...])
    const imgData = ctx.getImageData(0, 0, w, h);
    const data = imgData.data;

    // 將目前的 currentColor (例如 'rgb(255, 0, 0)') 轉為數值陣列
    const colorMatch = currentColor.match(/\d+/g);
    if (!colorMatch) return;
    const fillR = parseInt(colorMatch[0]);
    const fillG = parseInt(colorMatch[1]);
    const fillB = parseInt(colorMatch[2]);
    const fillA = 255; // 油漆桶填入實色

    // 取得起點像素的顏色 (目標顏色)
    const startPos = (startY * w + startX) * 4;
    const targetR = data[startPos];
    const targetG = data[startPos + 1];
    const targetB = data[startPos + 2];
    const targetA = data[startPos + 3];

    // 如果點擊的地方已經是我們要填的顏色，就提早結束 (避免死迴圈)
    if (targetR === fillR && targetG === fillG && targetB === fillB && targ
        return;
    }

    // 檢查某個像素是否符合目標顏色
    const matchColor = (pos) => {
        return data[pos] === targetR &&
               data[pos + 1] === targetG &&
               data[pos + 2] === targetB &&
               data[pos + 3] === targetA;
    };

    // 將像素上色
    const colorPixel = (pos) => {
        data[pos] = fillR;
        data[pos + 1] = fillG;
        data[pos + 2] = fillB;
        data[pos + 3] = fillA;
    };

    // 使用 Stack 儲存掃描點 (Scanline)
    const pixelStack = [[startX, startY]];
```

```javascript
function floodFill(startX, startY) {
    while (pixelStack.length > 0) {
        const newPos = pixelStack.pop();
        const x = newPos[0];
        let y = newPos[1];

        let currentPos = (y * w + x) * 4;

        // 1. 往上走到邊界，找到該欄的最頂端相應目標色
        while (y >= 0 && matchColor(currentPos)) {
            y--;
            currentPos -= w * 4;
        }

        // 退回一格回到有效像素
        y++;
        currentPos += w * 4;

        let reachLeft = false;
        let reachRight = false;

        // 2. 開始往下掃描並上色
        while (y < h && matchColor(currentPos)) {
            colorPixel(currentPos);

            // 檢查左側
            if (x > 0) {
                if (matchColor(currentPos - 4)) {
                    if (!reachLeft) {
                        pixelStack.push([x - 1, y]);
                        reachLeft = true;
                    }
                } else if (reachLeft) {
                    reachLeft = false;
                }
            }

            // 檢查右側
            if (x < w - 1) {
                if (matchColor(currentPos + 4)) {
                    if (!reachRight) {
                        pixelStack.push([x + 1, y]);
                        reachRight = true;
                    }
                } else if (reachRight) {
                    reachRight = false;
                }
            }

            // 往下走一格
            y++;
            currentPos += w * 4;
        }
    }
```

Refinement & Explanation: 看完了我覺得 ai 做的很棒

Scope of Usage 2: Image Editor Overlay (圖片臨時編輯模式)

Location: app.js, lines 681-805 ; style.css, lines 246-293

Prompt: "隔壁上下文太長開始出錯了 我接下來要做 User can adjust image size afterward 讓人可以上傳圖片並進入臨時編輯模式 然後直接合進畫布 所以不會有多個可編輯物件的問題"

Response:

```c
// ==========================================
// 臨時圖片編輯器系統 (Image Editor Mode)
// ==========================================
function initImageEditor(img) {
    const container = document.getElementById('canvas-container');
    const overlay = document.createElement('div');
    overlay.className = 'img-edit-overlay';

    // 1. 設定初始大小與位置 (限制最大尺寸並置中)
    let w = img.width, h = img.height;
    const maxW = container.clientWidth * 0.7; // 最大不超過畫布 70%
    const maxH = container.clientHeight * 0.7;
    if (w > maxW || h > maxH) {
        const ratio = Math.min(maxW / w, maxH / h);
        w *= ratio;
        h *= ratio;
    }

    let x = (container.clientWidth - w) / 2;
    let y = (container.clientHeight - h) / 2;

    overlay.style.width = `${w}px`;
    overlay.style.height = `${h}px`;
    overlay.style.left = `${x}px`;
    overlay.style.top = `${y}px`;

    // 2. 建立 DOM 元素
    const imgEl = document.createElement('img');
    imgEl.src = img.src;
    overlay.appendChild(imgEl);

    const handle = document.createElement('div');
    handle.className = 'resize-handle';
    overlay.appendChild(handle);

    const btn = document.createElement('button');
    btn.className = 'img-confirm-btn';
    btn.innerText = '✓ 完成 (Enter)';
    overlay.appendChild(btn);

    container.appendChild(overlay);

    // 3. 拖曳與縮放狀態變數
    let isDragging = false;
    let isResizing = false;
    let startX, startY, startW, startH, startLeft, startTop;
```

```c
    // 觸發拖曳
    overlay.addEventListener('mousedown', (e) => {
        if (e.target === handle || e.target === btn) return; // 點擊縮放點或按鈕時不觸發拖曳
        isDragging = true;
        startX = e.clientX;
        startY = e.clientY;
        startLeft = overlay.offsetLeft;
        startTop = overlay.offsetTop;
        e.stopPropagation();
    });

    // 觸發縮放
    handle.addEventListener('mousedown', (e) => {
        isResizing = true;
        startX = e.clientX;
        startY = e.clientY;
        startW = overlay.offsetWidth;
        startH = overlay.offsetHeight;
        e.stopPropagation();
    });

    // 滑鼠移動 (全域監聽避免滑太快掉幀)
    const onMouseMove = (e) => {
        if (isDragging) {
            const dx = e.clientX - startX;
            const dy = e.clientY - startY;
            overlay.style.left = `${startLeft + dx}px`;
            overlay.style.top = `${startTop + dy}px`;
        } else if (isResizing) {
            const dx = e.clientX - startX;
            const dy = e.clientY - startY;

            let newW = startW + dx;
            let newH = startH + dy;

            // 按住 Ctrl 等比例縮放
            if (e.ctrlKey) {
                const ratio = startW / startH;
                newH = newW / ratio;
            }

            overlay.style.width = `${Math.max(20, newW)}px`; // 限制最小寬度 20px
            overlay.style.height = `${Math.max(20, newH)}px`;
        }
    };

    const onMouseUp = () => {
        isDragging = false;
        isResizing = false;
    };
```

```c
    window.addEventListener('mousemove', onMouseMove);
    window.addEventListener('mouseup', onMouseUp);

    // 4. 確認並繪製到真正的 Canvas 上
    const confirmEdit = () => {
        saveState(); // 存檔支援 Undo

        const finalX = overlay.offsetLeft;
        const finalY = overlay.offsetTop;
        const finalW = overlay.offsetWidth;
        const finalH = overlay.offsetHeight;

        currentCtx.drawImage(img, finalX, finalY, finalW, finalH);

        // 清理 DOM 與事件監聽
        overlay.remove();
        window.removeEventListener('mousemove', onMouseMove);
        window.removeEventListener('mouseup', onMouseUp);
        document.removeEventListener('keydown', onKeyDown);

        // 自動切回游標工具
        document.getElementById('btn-cursor').click();
    };

    btn.addEventListener('click', confirmEdit);

    // 支援按 Enter 鍵快速確認
    const onKeyDown = (e) => {
        if (e.key === 'Enter') confirmEdit();
    };
    document.addEventListener('keydown', onKeyDown);
}
```

```c
/* ===== 圖片臨時編輯模式樣式 ===== */
.img-edit-overlay {
    position: absolute;
    border: 2px dashed #ff9800; /* 用亮色框選處於編輯狀態 */
    cursor: move;
    z-index: 1000;
    box-sizing: border-box;
}

.img-edit-overlay img {
    width: 100%;
    height: 100%;
    display: block;
    pointer-events: none; /* 讓滑鼠事件穿透圖片，由外層 overlay 統一接收拖曳 */
}

/* 右下角的縮放手柄 */
.resize-handle {
    position: absolute;
    width: 16px;
    height: 16px;
    background: white;
    border: 2px solid #1a80db;
    border-radius: 50%;
    right: -8px;
    bottom: -8px;
    cursor: nwse-resize;
    z-index: 1001;
}

/* 確認按鈕 */
.img-confirm-btn {
    position: absolute;
    top: -35px;
    right: -2px;
    background: #4caf50;
    color: white;
    border: none;
    border-radius: 5px;
    padding: 5px 10px;
    font-weight: bold;
    cursor: pointer;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2);
    white-space: nowrap;
}
.img-confirm-btn:hover {
    background: #45a049;
}
```

Refinement & Explanation: 加入「按住 Ctrl 鍵等比例縮放」、Enter 鍵確定圖片位置，提升 user experience

Scope of Usage 3: Dynamic Calligraphy Brush (動態書法畫筆)

Location: app.js, lines 112-139 (渲染邏輯) & lines 406-413 (切換邏輯)

Prompt: "我還想做書法畫筆 根據筆刷移動速度有寬度變化 透過點擊普通筆刷切換 type"

Response:

```
111     else if (currentTool === 'brush') {
112         currentCtx.globalCompositeOperation = 'source-over';
113         currentCtx.strokeStyle = currentColor;
114
115         // 👇 新增：判斷是普通畫筆還是書法模式
116         if (brushType === 'calligraphy') {
117             const now = Date.now();
118             const dt = Math.max(1, now - lastTime); // 避免除以 0
119             const dx = x - lastX;
120             const dy = y - lastY;
121             const dist = Math.sqrt(dx * dx + dy * dy);
122             const velocity = dist / dt; // 計算速度
123
124             // 速度越快，筆觸越細 (減去速度乘上一個靈敏度係數)
125             const targetWidth = Math.max(1, currentSize - velocity * 4);
126
127             // 平滑過渡 (Easing)：上一點寬度佔 70%，目標寬度佔 30%
128             const lineWidth = lastLineWidth * 0.7 + targetWidth * 0.3;
129
130             currentCtx.lineWidth = lineWidth;
131
132             // 更新狀態供下一個 Frame 使用
133             lastLineWidth = lineWidth;
134             lastTime = now;
135             lastX = x;
136             lastY = y;
137         } else {
138             currentCtx.lineWidth = currentSize;
139         }
```

```
406         if (btn.id === 'btn-brush' && currentTool === 'brush') {
407             // 已經是畫筆狀態下再次點擊，觸發切換
408             brushType = brushType === 'normal' ? 'calligraphy' : 'normal';
409             // 替換按鈕圖示與提示
410             btn.innerText = brushType === 'calligraphy' ? '🖌️' : '🖌';
411             btn.title = brushType === 'calligraphy' ? 'Brush (書法模式)' :
412             return; // 切換完就跳出，不需要跑後面的重置邏輯
413         }
```

Refinement & Explanation: 根據實際測試手感調整了 Easing (平滑過渡) 的權重參數 (上一點寬度佔 70%，目標寬度佔 30%)、速度對應寬度的靈敏度係數、也將此模式的切換綁定至 UI 按鈕，因為我覺得這樣看起來比較順

Scope of Usage: Segment 4 – 調色盤

Location: app.js, lines 493-623

Prompt: (範例圖片) “幫我做調色盤 有 rgb 滑桿”

Response:

```
493 // ========================================
494 // 5. 調色盤系統 (Color Picker)
495 // ========================================
496 const colorBlock = document.getElementById('color-block');
497 const blockCtx = colorBlock.getContext('2d', { willReadFrequently: true });
498 const colorStrip = document.getElementById('color-strip');
499 const stripCtx = colorStrip.getContext('2d', { willReadFrequently: true });
500
501 let baseColor = currentColor;
502 let pickerX = 120, pickerY = 0;
503
504 function drawColorStrip() {
505     stripCtx.clearRect(0, 0, colorStrip.width, colorStrip.height);
506     const gradient = stripCtx.createLinearGradient(0, 0, 0, colorStrip.height);
507     gradient.addColorStop(0, 'rgb(255, 0, 0)'); gradient.addColorStop(0.17, 'rgb(255, 255,
508     gradient.addColorStop(0.34, 'rgb(0, 255, 0)'); gradient.addColorStop(0.51, 'rgb(0, 255
509     gradient.addColorStop(0.68, 'rgb(0, 0, 255)'); gradient.addColorStop(0.85, 'rgb(255, 0
510     gradient.addColorStop(1, 'rgb(255, 0, 0)');
511     stripCtx.fillStyle = gradient; stripCtx.fillRect(0, 0, colorStrip.width, colorStrip.height
512 }
513
514 function drawColorBlock(showMarker = true) {
515     blockCtx.clearRect(0, 0, colorBlock.width, colorBlock.height);
516     blockCtx.fillStyle = baseColor; blockCtx.fillRect(0, 0, colorBlock.width, colorBlock.height
517
518     const whiteGrad = blockCtx.createLinearGradient(0, 0, colorBlock.width, 0);
519     whiteGrad.addColorStop(0, 'rgba(255,255,255,1)'); whiteGrad.addColorStop(1, 'rgba(255,
520     blockCtx.fillStyle = whiteGrad; blockCtx.fillRect(0, 0, colorBlock.width, colorBlock.height
521
522     const blackGrad = blockCtx.createLinearGradient(0, 0, 0, colorBlock.height);
523     blackGrad.addColorStop(0, 'rgba(0,0,0,0)'); blackGrad.addColorStop(1, 'rgba(0,0,0,1)')
524     blockCtx.fillStyle = blackGrad; blockCtx.fillRect(0, 0, colorBlock.width, colorBlock.height
525
526     if (showMarker) {
527         blockCtx.beginPath(); blockCtx.arc(pickerX, pickerY, 4, 0, Math.PI * 2);
528         blockCtx.strokeStyle = 'white'; blockCtx.lineWidth = 1.5; blockCtx.stroke();
529         blockCtx.beginPath(); blockCtx.arc(pickerX, pickerY, 5, 0, Math.PI * 2);
530         blockCtx.strokeStyle = 'black'; blockCtx.lineWidth = 1; blockCtx.stroke();
531     }
532 }
533
534 drawColorStrip(); drawColorBlock();
```

```
536 function applySelectedColor(r, g, b) {
537     currentColor = `rgb(${r},${g},${b})`;
538     document.getElementById('color-preview').style.backgroundColor = currentColor;
539     document.getElementById('color-text').innerText = currentColor;
540     document.getElementById('slider-r').value = r;
541     document.getElementById('slider-g').value = g;
542     document.getElementById('slider-b').value = b;
543
544     // 改寫自定畫筆全局顏色，讓其他色盤點完成時變動
545     if (currentTool === 'cursor') document.getElementById('btn-brush').click();
546 }
547
548 let isPickingBlock = false, isPickingStrip = false, animationFrameId = null;
549
550 function pickColor(clientX, clientY, ctx, canvas, isStrip) {
551     const rect = canvas.getBoundingClientRect();
552     const x = Math.floor(Math.max(0, Math.min(clientX - rect.left, canvas.width - 1)));
553     const y = Math.floor(Math.max(0, Math.min(clientY - rect.top, canvas.height - 1)));
554
555     if (isStrip) {
556         const pixel = ctx.getImageData(x, y, 1, 1).data;
557         baseColor = `rgba(${pixel[0]}, ${pixel[1]}, ${pixel[2]}, 1)`;
558         drawColorBlock(true);
559         applySelectedColor(pixel[0], pixel[1], pixel[2]);
560     } else {
561         pickerX = x; pickerY = y;
562         drawColorBlock(false);
563         const pixel = ctx.getImageData(x, y, 1, 1).data;
564         applySelectedColor(pixel[0], pixel[1], pixel[2]);
565         drawColorBlock(true);
566     }
567 }
568
569 function handleMouseMove(e, ctx, canvas, isStrip) {
570     const clientX = e.clientX, clientY = e.clientY;
571     if (animationFrameId) cancelAnimationFrame(animationFrameId);
572     animationFrameId = requestAnimationFrame(() => pickColor(clientX, clientY, ctx, canvas, is
573 }
574
575 colorBlock.addEventListener('mousedown', (e) => { isPickingBlock = true; pickColor(e.clientX, e
576 colorStrip.addEventListener('mousedown', (e) => { isPickingStrip = true; pickColor(e.clientX, e
577
578 window.addEventListener('mousemove', (e) => {
579     if (isPickingBlock) handleMouseMove(e, blockCtx, colorBlock, false);
580     if (isPickingStrip) handleMouseMove(e, stripCtx, colorStrip, true);
581 });
582
583 window.addEventListener('mouseup', () => {
584     isPickingBlock = false; isPickingStrip = false;
585     if (animationFrameId) cancelAnimationFrame(animationFrameId);
586 });
```

```
588 function rgbToHsv(r, g, b) {
589     r /= 255; g /= 255; b /= 255;
590     const max = Math.max(r, g, b), min = Math.min(r, g, b), d = max - min;
591     let h = 0, s = (max === 0 ? 0 : d / max), v = max;
592     if (max !== min) {
593         if (max === r) h = (g - b) / d + (g < b ? 6 : 0);
594         else if (max === g) h = (b - r) / d + 2;
595         else if (max === b) h = (r - g) / d + 4;
596         h /= 6;
597     }
598     return { h: h * 360, s: s, v: v };
599 }
600
601 [document.getElementById('slider-r'), document.getElementById('slider-g'), document.getElement
602     slider.addEventListener('input', () => {
603         const r = parseInt(document.getElementById('slider-r').value);
604         const g = parseInt(document.getElementById('slider-g').value);
605         const b = parseInt(document.getElementById('slider-b').value);
606
607         applySelectedColor(r, g, b);
608         const hsv = rgbToHsv(r, g, b);
609         baseColor = `hsl(${hsv.h}, 100%, 50%)`;
610         pickerX = hsv.s * colorBlock.width;
611         pickerY = (1 - hsv.v) * colorBlock.height;
612         drawColorBlock();
613     });
614 });
615
616 // 同步更新調色盤 UI 的小幫手
617 function updateColorPickerUI(r, g, b) {
618     const hsv = rgbToHsv(r, g, b);
619     baseColor = `hsl(${hsv.h}, 100%, 50%)`;
620     pickerX = hsv.s * colorBlock.width;
621     pickerY = (1 - hsv.v) * colorBlock.height;
622     drawColorBlock(true);
623 }
```

Refinement & Explanation: line 526-531 加小圈圈標示目前顏色，讓使用者更掌握色調

Scope of Usage: Segment 5 - Anti-Flickering Shapes

Location: app.js (Lines 50-78, 146-169, 555-645)

Prompt: “圖形工具”,”按著 ctrl 鎖定正圓 長方形也幫我改”

Response:

```
50  // ==========================================
51  // 3. 繪圖核心系統 (Drawing Core)
52  // ==========================================
53  function startPosition(e) {
54      // 如果是選擇工具，直接中斷，不執行任何繪圖動作
55      if (currentTool === 'cursor') return;
56
57      saveState();
58      isDrawing = true;
59
60      // 👇 使用清爽的相對座標 offsetX / offsetY
61      startX = e.offsetX;
62      startY = e.offsetY;
63
64      snapshot = currentCtx.getImageData(0, 0, canvases[activeLayerIndex].width, canvases[activel
65
66      if (currentTool === 'brush' || currentTool === 'eraser') {
67          currentCtx.beginPath();
68          currentCtx.moveTo(startX, startY);
69
70          // 👇 新增：紀錄初始狀態，供畫法畫筆計算速度用
71          lastX = startX;
72          lastY = startY;
73          lastTime = Date.now();
74          lastLineWidth = currentSize;
75
76          draw(e);
77      }
78  }
```

```
146     else if (currentTool === 'rect') {
147         let w = x - startX, h = y - startY;
148         if (e.ctrlKey) {
149             const size = Math.max(Math.abs(w), Math.abs(h));
150             w = w > 0 ? size : -size;
151             h = h > 0 ? size : -size;
152         }
153         currentCtx.rect(startX, startY, w, h);
154         currentCtx.stroke();
155     }
156     else if (currentTool === 'circle') {
157         let w = x - startX, h = y - startY;
158         if (e.ctrlKey) {
159             const size = Math.max(Math.abs(w), Math.abs(h));
160             const radius = size / 2;
161             const centerX = startX + (w > 0 ? radius : -radius);
162             const centerY = startY + (h > 0 ? radius : -radius);
163             currentCtx.arc(centerX, centerY, radius, 0, 2 * Math.PI);
164             currentCtx.stroke();
165         } else {
166             currentCtx.ellipse(startX + w / 2, startY + h / 2, Math.abs(w) / 2, Math.abs(h) / 2
167             currentCtx.stroke();
168         }
169     }
```

Refinement & Explanation: 圓拖曳範圍從半徑改為外接方框，熟悉的 powerpoint 是這樣做的

Scope of Usage: Segment 6 - Undo/Redo & Text/Layer Management

Location: app.js (Lines 30-47, 625-679) & index.html (Lines 74-105)

Prompt: “undo/redo”,” 想做 layer, text”

Response: 提供二維陣列堆疊管理多圖層歷史紀錄的邏輯，以及在畫布上動態生成 input 標籤以輸入文字的做法。

```
30  // ==========================================
31  // 2. 歷史紀錄系統 (Undo / Redo)
32  // ==========================================
33  let undoStacks = [[], []];
34  let redoStacks = [[], []];
35  const MAX_HISTORY = 50;
36
37  function saveState() {
38      const canvas = canvases[activeLayerIndex];
39      const ctx = contexts[activeLayerIndex];
40      const currentState = ctx.getImageData(0, 0, canvas.width, canvas.height);
41
42      undoStacks[activeLayerIndex].push(currentState);
43      if (undoStacks[activeLayerIndex].length > MAX_HISTORY) {
44          undoStacks[activeLayerIndex].shift();
45      }
46      redoStacks[activeLayerIndex] = [];
47  }
```

```
625 // ==========================================
626 // 6. 系統按鈕與圖層管理 (System & Layers)
627 // ==========================================
628 document.getElementById('btn-refresh').addEventListener('click', () => {
629     saveState();
630     contexts.forEach((ctx, idx) => ctx.clearRect(0, 0, canvases[idx].width, canvases[idx].heig
631 });
632
633 document.getElementById('btn-undo').addEventListener('click', () => {
634     if (undoStacks[activeLayerIndex].length > 0) {
635         const currentState = currentCtx.getImageData(0, 0, canvases[activeLayerIndex].width, c
636         redoStacks[activeLayerIndex].push(currentState);
637         currentCtx.putImageData(undoStacks[activeLayerIndex].pop(), 0, 0);
638     }
639 });
640
641 document.getElementById('btn-redo').addEventListener('click' ' => ' ' '
```

Scope of Usage: Segment 8 - 整理
Location: (all the code)
Prompt: (all the code) 幫我整理跟美化一下
Response: (all the code)
Refinement & Explanation: 看完了我覺得 ai 做的很棒，後來再微調色彩